

numpyexercises

January 23, 2026

1 Homework 1: Programming Component

This notebook contains work for problems 5 and 6 of homework 1. After completing these problems, please make sure you have run your notebook from beginning to end (Kernel → Restart kernel and run all cells), and that all figures, plots, and values are embedded in your submitted .ipynb file.

```
[1]: %matplotlib inline
```

```
[2]: import matplotlib.pyplot as plt
import numpy as np
```

2 Problem 5: Audio Compression

```
[ ]:
```

```
[4]: ### Import additional packages in this cell ###
from scipy.io import wavfile
from math import floor
import sounddevice
```

```
[74]: sr, audio = wavfile.read('sample.wav')

F = np.fft.fft(audio)

thirds = floor(len(F)/3)
center = floor(len(F)/2)

compressed = F.copy()
compressed[center-thirdrs:center] = 0
compressed[center:center+thirds] = 0

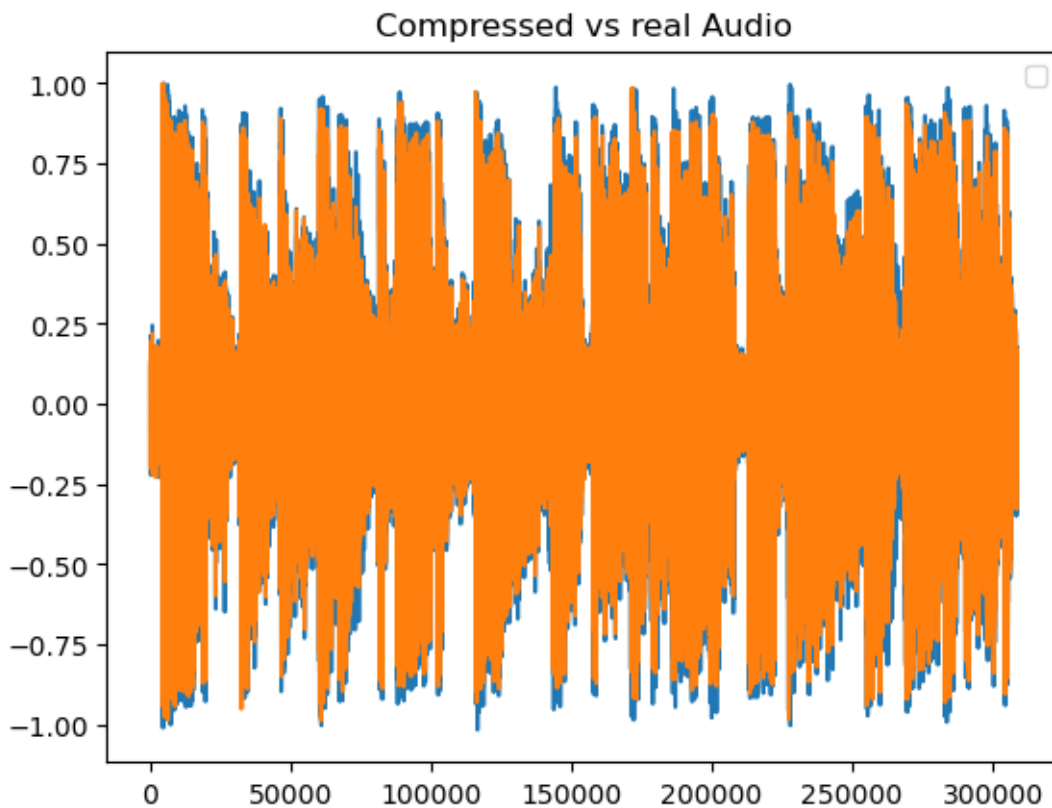
compressedaudio = np.real(np.fft.ifft(compressed)) # why did it take so long to
↪ realize this needed to be real TT
```

```
audio = audio/max(audio)
compressedaudio = compressedaudio/(max(compressedaudio))
```

```
[256]: plt.plot(audio,label="real")

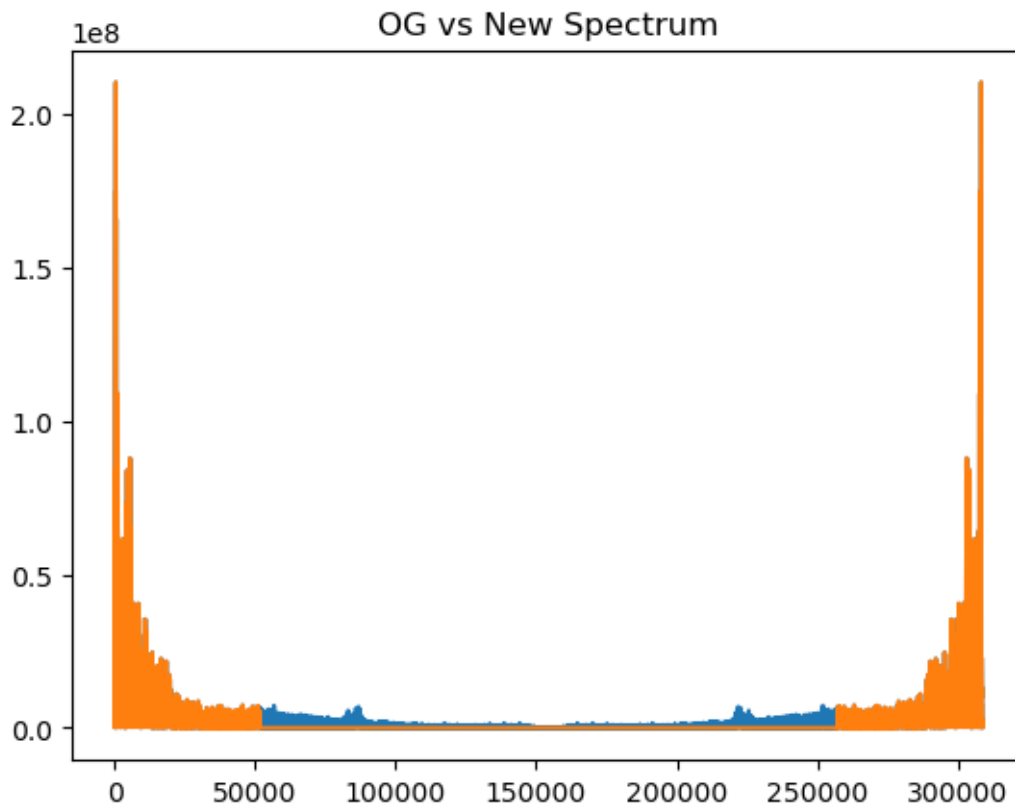
plt.plot(compressedaudio,label="compressed")
plt.legend("o","c")
plt.title("Compressed vs real Audio")
plt.show()
```

C:\Users\Sorin\AppData\Local\Temp\ipykernel_6116\562237648.py:4: UserWarning:
 Legend does not support handles for str instances.
 A proxy artist may be used instead.
 See:
https://matplotlib.org/stable/users/explain/axes/legend_guide.html#controlling-the-legend-entries
 plt.legend("o","c")



```
[254]: plt.plot(abs(F))
plt.plot(abs(compressed))
plt.title("OG vs New Spectrum")
```

```
plt.show()
```



```
[76]: sounddevice.play(audio,sr)
```

```
[77]: sounddevice.play(compressedaudio,sr)
```

3 Problem 6: Numpy Exercises

After going through the `ipythonintro.ipynb` tutorial, please complete the following exercises. For this assignment, you should not use any generative AI services (ChatGPT, Github Co-pilot, or equivalent) to generate code. You may use any online documentation (e.g. numpy) and stack overflow to help debug.

3.1 Problem 1

Perform the following steps, each on one line of code: - Create a variable `a` that contains a single integer element 1 - Append the elements 1, 3, 3, 5, and 5 to `a` (you can do this with either the `append` function or the `extend` function) - Convert `a` from a python list to a numpy array - Reshape the array to have 2 columns and 3 rows - Multiply all elements by 2 - Display the resulting matrix

Note that some operations are done in place, and others require you to re-assign the result. Make

you sure you know which operations do which!

```
[84]: a = [1]
      a.extend([1,3,3,5,5])
      nparr = np.array(a)
      newshape = nparr.reshape(3,2)
      newshape = newshape * 2
      print(newshape)
```

```
[[ 2  2]
 [ 6  6]
 [10 10]]
```

3.2 Problem 2

Do the following: - Create a function called `double_plus_one` that doubles the input, adds one, and returns the result. - Using an initial input of 1, call the function on itself 10 times using a for loop. - Print the result using an f-string that says **The result is XX** where XX specifies the result. Read up on f-strings if you are not familiar with them.

```
[92]: def double_plus_one(inp):
      res = inp * 2 + 1
      return res

      val = 1
      for i in range(10):
          val = double_plus_one(val)

      print(f"end: {val}")
```

end: 2047

3.3 Problem 3

Complete the exercise from `ipythonintro.ipynb`: Generate an $N \times N$ matrix whose element values are the sum of the row and column indices. First do this with a double nested for loop, then with a single for loop, then with no for loops (only broadcasting).

```
[93]: def getMatrix_bad(N):
      A = np.zeros((N,N))
      for i in range(N):
          for j in range(N):
              A[i,j] = i + j
      return A
```

```
[94]: A = getMatrix_bad(4) # verify
      A
```

```
[94]: array([[0., 1., 2., 3.],
            [1., 2., 3., 4.],
            [2., 3., 4., 5.],
            [3., 4., 5., 6.]])
```

```
[95]: def getMatrix_better(N):
      A = np.zeros((N,N))
      for i in range(N):
          A[i,:] = i + np.arange(N)
      return A
```

```
[96]: A = getMatrix_better(4) # verify
      A
```

```
[96]: array([[0., 1., 2., 3.],
            [1., 2., 3., 4.],
            [2., 3., 4., 5.],
            [3., 4., 5., 6.]])
```

```
[ ]:
```

```
[ ]: array([[1, 4],
            [2, 5],
            [3, 6]])
```

```
[149]: def getMatrix_best(N):
      return np.arange(N).reshape(-1,1) + np.arange(N)
```

```
[150]: A = getMatrix_best(4) # verify
      A
```

```
[150]: array([[0, 1, 2, 3],
            [1, 2, 3, 4],
            [2, 3, 4, 5],
            [3, 4, 5, 6]])
```

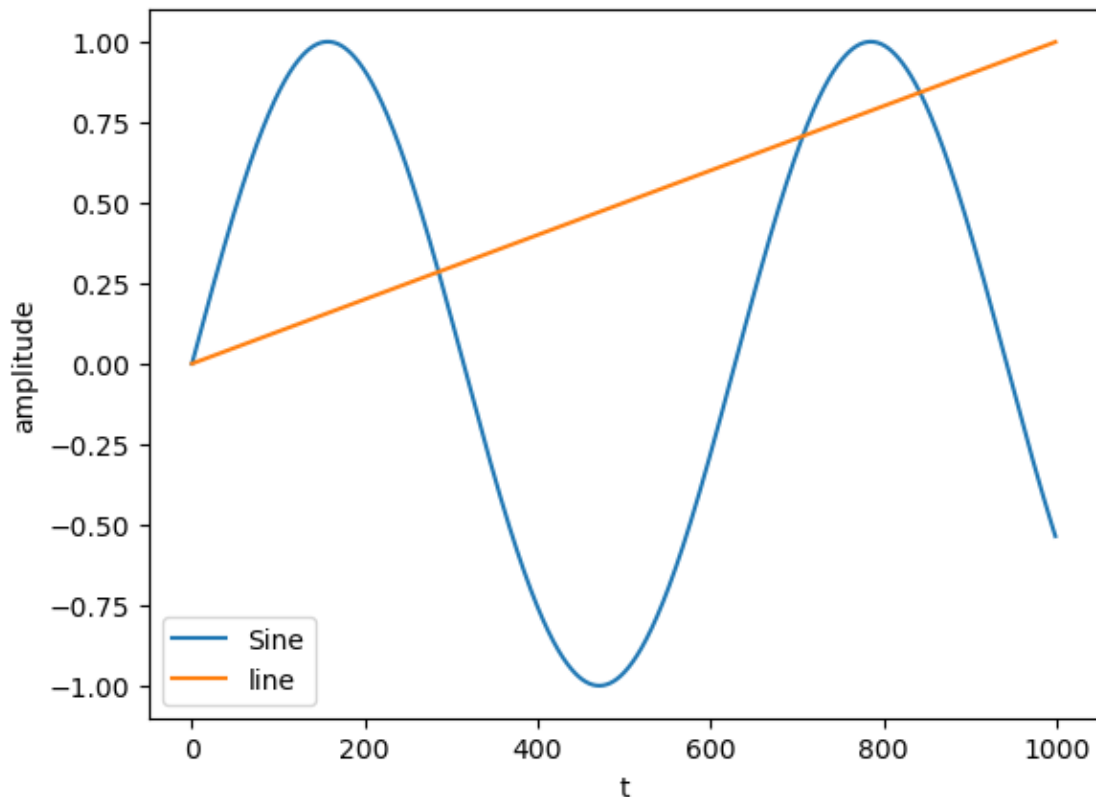
3.4 Problem 4

Do the following: - Plot the function $x = \sin(t)$ using values of t between 0 and 10 with a spacing of 0.01. Use a blue line to connect the samples. The function `numpy.arange` may be helpful. - On the same graph, overlay a plot of $y = t*0.1$ using the same values of t . Plot the line in red. - Label the x-axis as 't' - Add a legend to indicate which variable is being shown, as well as a title called 'Problem 4'. Render your plot in the notebook.

```
[ ]: x = np.arange(0,10,0.01)

plt.plot(np.sin(x),label="Sine")
```

```
plt.plot(0.1*x,label="line")
plt.xlabel("t")
plt.ylabel("amplitude")
plt.legend()
plt.show()
```



3.5 Problem 5

Do the following: - Implement a `myDFT` function that takes a 1-D array as input and returns the DFT of the input. Your function can be implemented as a double for loop for now (in the next assignment you will re-implement it in a vectorized way). - Create a random input of length 2000 using the `np.random.rand` command. Compare the output of your `myDFT` function to the `numpy.fft.fft` function. Verify that they match using the `numpy.allclose` function. - Measure the runtime of your DFT function using the `%timeit` magic command, and compare the runtime to that of the `np.fft.fft` function. Print your measured runtimes. How many times faster is the numpy implementation than your unoptimized version?

```
[248]: def myDFT(x):
        X = np.array([])
        N = len(x)
```

```

for k in range(N):
    val = 0
    for n in range(N):
        val = val + x[n] * np.exp(-1j * n * 2 * np.pi / N * k )
    X = np.append(X, val)
return X

```

```
[249]: rnd = np.random.rand(2000)
```

```

bad = myDFT(rnd)
good = np.fft.fft(rnd)

```

```
[245]: len(bad)
```

```
[245]: 2000
```

```
[250]: quality = np.allclose(good, bad, rtol=1e-03, atol=1e-04)
quality
```

```
[250]: True
```

```
[253]: plt.plot(bad)
plt.plot(good)
plt.xlabel("K")
plt.ylabel("Frequency Amplitude")
plt.title("Foruier comparison manual vs numpy")
plt.show

```

```

c:\Users\Sorin\anaconda3\envs\E207_Spr26\Lib\site-
packages\matplotlib\cbook.py:1719: ComplexWarning: Casting complex values to
real discards the imaginary part
    return math.isfinite(val)
c:\Users\Sorin\anaconda3\envs\E207_Spr26\Lib\site-
packages\matplotlib\cbook.py:1355: ComplexWarning: Casting complex values to
real discards the imaginary part
    return np.asarray(x, float)

```

```
[253]: <function matplotlib.pyplot.show(close=None, block=None)>
```

